

Encoders

Encoders are the next major concept after basic drive.

Once a robot can drive manually, the team should learn how the robot measures motion. Encoders are sensors built into many FTC motors that count shaft rotation. Code can use those counts to estimate distance, speed, or mechanism position.

The core idea is:

encoder ticks -> motor rotation -> wheel/mechanism movement

Why Encoders Matter

Without encoders, code usually commands motor power for a fixed time:

drive at 50 percent power for 1 second

That is easy, but not very repeatable. The robot may move farther or shorter depending on:

- battery voltage
- carpet friction
- robot weight
- wheel slip
- motor load
- gear ratio
- whether the robot hits a game piece or wall

With encoders, code can ask for motion in measured units:

drive until the motor has turned 1120 ticks
move the slide to 900 ticks
spin the turret to 300 ticks
hold shooter speed at 2800 RPM

Encoders are the bridge between software and physical motion.

What An Encoder Measures

An encoder does not directly measure inches, centimeters, or degrees.

It measures ticks.

For a motor shaft:

ticks -> motor shaft rotations

For a wheel:

motor shaft rotations -> wheel rotations -> distance traveled

For a mechanism:

motor shaft rotations -> gear output rotations -> arm/slide/turret movement

Basic Formulas

Ticks To Motor Rotations

$\text{motorRotations} = \text{encoderTicks} / \text{ticksPerMotorRotation}$

For many FTC motors, the base encoder value is often treated as 28 ticks per motor shaft rotation before gearbox reduction. The effective ticks at the output shaft depend on gearbox ratio.

Ticks Per Output Rotation

$\text{ticksPerOutputRotation} = \text{ticksPerMotorRotation} * \text{gearRatio}$

Example:

$\text{ticksPerMotorRotation} = 28$
 $\text{gearRatio} = 19.2$

$\text{ticksPerOutputRotation} = 28 * 19.2 = 537.6 \text{ ticks}$

That means about 537.6 encoder ticks for one output shaft rotation.

Wheel Distance Per Rotation

$\text{wheelCircumference} = \text{wheelDiameter} * \pi$

If the wheel diameter is 96 mm:

$\text{wheelCircumference} = 96 * \pi = \text{about } 301.6 \text{ mm}$

Ticks To Distance

$\text{distance} = \text{encoderTicks} / \text{ticksPerOutputRotation} * \text{wheelCircumference}$

Distance To Ticks

$\text{targetTicks} = \text{distance} / \text{wheelCircumference} * \text{ticksPerOutputRotation}$

This is the formula used for "drive forward a fixed distance."

FTC Motor Modes

FTC motors can be used in several modes. New teams should understand these before writing autonomous code.

RUN_WITHOUT_ENCODER

The motor runs based only on power.

`setPower(0.5)`

Use this for:

- basic TeleOp drive
- mechanisms that do not need position control
- first bring-up tests

The encoder may still be readable, but the motor controller is not using it to regulate speed.

RUN_USING_ENCODER

The motor controller uses encoder feedback to regulate velocity better.

Use this for:

- shooter motors
- drivetrain velocity experiments
- mechanisms where smoother speed matters

You still command power or velocity, depending on the code style.

RUN_TO_POSITION

The motor controller drives toward a target encoder position.

Typical pattern:

```
motor.setTargetPosition(targetTicks);  
motor.setMode(DcMotor.RunMode.RUN_TO_POSITION);  
motor.setPower(0.5);
```

Use this for:

- arms
- slides
- turrets
- simple autonomous drive-distance exercises

Important: RUN_TO_POSITION needs a target position and nonzero power.

STOP_AND_RESET_ENCODER

Resets the encoder count to zero.

Typical pattern:

```
motor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);  
motor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
```

Use this carefully. If you reset at the wrong time, the code loses its reference point.

For mechanisms, reset should usually happen only at a known physical position, such as a limit switch or hard stop.

Simple Exercise 1: Read Encoder Ticks

Goal:

Learn how encoder values change when a motor moves.

Steps:

1. Put the robot on blocks.
2. Reset one drive motor encoder.
3. Display encoder ticks in telemetry.
4. Run the motor slowly.

5. Observe whether ticks increase or decrease.
6. Reverse motor power and observe the sign.

Questions for students:

- Does forward power make ticks increase or decrease?
- Is the sign the same on the left and right sides?
- Does reversing motor direction also reverse encoder sign?
- What happens when the motor is unplugged?

Expected learning:

Encoder signs matter. If a motor or encoder sign is backwards, autonomous distance and odometry math will be wrong.

Simple Exercise 2: Turn A Motor To A Fixed Position

Goal:

Use encoder ticks as a position target.

Example:

```
motor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);  
motor.setTargetPosition(500);  
motor.setMode(DcMotor.RunMode.RUN_TO_POSITION);  
motor.setPower(0.3);
```

Steps:

1. Put the robot on blocks.
2. Command a motor to 500 ticks.
3. Watch current position in telemetry.
4. Confirm the motor stops near the target.
5. Try 1000 ticks.
6. Try -500 ticks.

Questions for students:

- Did the motor move the expected direction?
- Did it stop close to the target?
- Did it overshoot?
- What happens if power is too low?
- What happens if power is too high?

Expected learning:

Encoder position control is useful, but target direction, motor direction, and power still matter.

Simple Exercise 3: Drive Forward A Fixed Distance

Goal:

Convert a desired distance into encoder ticks.

Formula:

$$\text{targetTicks} = \text{distance} / \text{wheelCircumference} * \text{ticksPerOutputRotation}$$

Example assumptions:

wheelDiameter = 96 mm
gearRatio = 19.2
ticksPerMotorRotation = 28
distance = 600 mm

Calculations:

wheelCircumference = $96 * \pi = 301.6$ mm
ticksPerOutputRotation = $28 * 19.2 = 537.6$ ticks
targetTicks = $600 / 301.6 * 537.6 = \text{about } 1069$ ticks

Steps:

1. Measure wheel diameter.
2. Find or estimate gear ratio.
3. Calculate target ticks.
4. Reset drive encoders.
5. Command all drive motors to the target.
6. Drive at moderate power.
7. Stop when motors reach target.
8. Measure actual distance traveled.

Questions for students:

- Did the robot drive straight?
- Did all motors reach the same target?
- Was the measured distance close to expected?
- Did wheel slip affect the result?
- Was the stopping behavior smooth?

Expected learning:

The math gives a starting point, but the real robot still needs calibration.

Simple Exercise 4: Measure Actual vs Expected Distance

Goal:

Teach calibration and error measurement.

Create a table:

Trial	Target distance	Target ticks	Actual distance	Error
1	600 mm	1069	580 mm	-20 mm
2	600 mm	1069	585 mm	-15 mm
3	600 mm	1069	578 mm	-22 mm

Then ask:

average actual distance = ?

average error = ?

correction factor = target distance / average actual distance

If the robot always drives short, increase the effective wheel diameter correction or ticks-per-distance conversion.

If the robot always drives long, decrease it.

Expected learning:

Engineering is measurement and adjustment. Constants should come from the real robot, not only from the product page.

Simple Exercise 5: Gear Ratio And Wheel Diameter

Goal:

Show why mechanical details affect software.

Change one assumption at a time:

- wheel diameter
- gear ratio
- ticks per motor rotation
- target distance

Have students calculate how target ticks change.

Example:

same distance, smaller wheel -> more wheel rotations -> more ticks

same distance, larger wheel -> fewer wheel rotations -> fewer ticks

higher gear reduction -> more motor rotations per wheel rotation -> more ticks

lower gear reduction -> fewer motor rotations per wheel rotation -> fewer ticks

Expected learning:

Code is not separate from mechanical design. Wheel size and gearing directly change encoder math.

Mechanism Examples

Encoders are not only for drive.

Linear Slide

motor ticks -> spool rotations -> string/belt movement -> slide height

Useful target:

slideHeightMm -> targetTicks

Arm

motor ticks -> gear output rotation -> arm angle

Useful target:

armAngleDegrees -> targetTicks

Turret

motor ticks -> turret gear rotation -> turret angle

Useful target:

turretAngleRadians -> targetTicks

Shooter

encoder ticks per second -> wheel RPM

Useful target:

targetRPM -> motor velocity

Homing And Zero Positions

Encoder counts are relative.

When the robot powers on, a mechanism encoder may not know where the mechanism physically is. For example, an arm might start halfway up, but code may think position zero is wherever the motor happened to boot.

For mechanisms, use a known reference:

- limit switch
- touch sensor
- magnetic sensor
- hard stop, used carefully

Homing pattern:

slowly move toward the known limit

when sensor is pressed:

stop motor

reset encoder to zero

Then all later positions are relative to a real physical reference.

Do not rely on encoder reset alone unless the mechanism is already at a known position.

Telemetry To Add

For encoder work, telemetry should show:

- current encoder position
- target encoder position
- motor power
- motor mode
- whether the motor is busy
- calculated target distance or angle
- measured error

Example:

```
front_left position: 842  
front_left target: 1069  
front_left busy: true
```

Good telemetry makes encoder problems much faster to debug.

Common Problems

Motor Moves The Wrong Direction

Likely causes:

- motor direction reversed
- target sign wrong
- encoder sign assumption wrong

Fix:

- test one motor at a time
- verify positive power direction
- verify positive encoder direction

Robot Does Not Drive Straight

Likely causes:

- left and right motors have different directions
- one motor is weaker or dragging
- wheel slip
- mechanical friction
- target positions are not mirrored correctly

Fix:

- test each wheel on blocks
- compare encoder values
- reduce speed
- check drivetrain mechanically

Motor Never Reaches Target

Likely causes:

- power too low
- mechanism jammed
- target unreachable
- motor mode wrong
- encoder cable problem

Fix:

- show current position in telemetry
- confirm encoder value changes
- confirm target position is reasonable
- test with a smaller movement

Distance Is Consistently Wrong

Likely causes:

- wheel diameter wrong
- gear ratio wrong
- ticks per rotation wrong
- wheel slip

Fix:

- measure actual wheel diameter
- measure actual travel over several trials
- apply a correction factor

Safety Rules

Always start encoder exercises safely.

- Put the robot on blocks for first tests.
- Use low motor power at first.
- Test one motor before testing all motors.
- Keep hands away from wheels and mechanisms.
- Add a cancel/stop button for mechanisms.
- Never run a slide, arm, or turret to a target before verifying limits.

Suggested Learning Order

Use this order for a new team:

1. Read encoder ticks in telemetry.
2. Run one motor and observe encoder direction.
3. Reset encoder and move to a fixed tick target.
4. Convert wheel distance to ticks.
5. Drive forward a measured distance.
6. Measure actual vs expected movement.

7. Tune constants.
8. Apply the same ideas to a mechanism.
9. Add homing with a limit switch.
10. Use encoders as feedback for autonomous.

Key Takeaways

Encoders let the robot measure movement.

The essential chain is:

encoder ticks -> motor rotation -> wheel/mechanism movement

Students should understand:

- ticks are not distance by themselves
- gear ratio changes ticks per output rotation
- wheel diameter changes distance per rotation
- encoder signs matter
- real robots need calibration
- telemetry is required for debugging
- mechanisms need known zero positions

Once the team understands encoders, they are ready for PID, simple autonomous movement, and eventually odometry/localization.